TRIVAXA / TRISECMED

System ewidencji i zarządzania aparaturą medyczną

SPECYFIKACJA TECHNICZNA BACKENDU

Backend Architecture & API Specification Document

Stack:
C# · ASP.NET Core 8 · Entity Framework Core · PostgreSQL 16
Hangfire · FluentValidation · MediatR · Docker · Debian

mgr inż. Mateusz Bartoszewicz · mgr inż. Stanisław Rachwał

Wersja 1.0 · Kwiecień 2026 · DOKUMENT POUFNY

1. Cel i zakres dokumentu

Cel: Niniejszy dokument opisuje kompletną architekturę techniczną warstwy backendowej systemu Trivaxa / Trisecmed. Dokument stanowi podstawę do implementacji i nie zakłada żadnego konkretnego rozwiązania frontendowego — backend projektowany jest jako niezależne REST API, do którego może być podłączony dowolny klient (przeglądarka, aplikacja mobilna, narzędzie zewnętrzne).

Zakres dokumentu:

- Architektura systemu i podział na moduły
- Stos technologiczny i uzasadnienie wyboru C# / ASP.NET Core 8
- Struktura projektu i wzorce projektowe
- Model danych — encje, relacje, schemat PostgreSQL
- Specyfikacja REST API — endpointy, kontrakty, kody odpowiedzi
- System uwierzytelniania i autoryzacji (JWT + RBAC)
- Kolejka zadań i powiadomienia (Hangfire)
- Infrastruktura — Docker Compose na Debianie
- Strategia backupu i monitoringu
- Harmonogram implementacji backendu

Ważne	Frontend nie jest przedmiotem tego dokumentu. Decyzja o technologii frontendowej zostanie podjęta po zakończeniu implementacji backendu i weryfikacji złożoności widoków w praktyce.

2. Architektura systemu

2.1 Przegląd

Backend systemu Trivaxa zbudowany jest jako monolityczna aplikacja z wyraźnym wewnętrznym podziałem na moduły (modular monolith). To świadoma decyzja architektoniczna dla zespołu dwuosobowego — unikamy złożoności mikroserwisów przy zachowaniu czystego podziału odpowiedzialności.

Wzorzec	Zastosowanie	Uzasadnienie
Modular Monolith	Podział systemu na samodziealnie moduły	Prostota deploymentu, łatwość debugowania, brak network overhead
CQRS (via MediatR)	Rozdzielenie komend (zapis) od zapytań (odczyt)	Czytelność kodu, łatwe testowanie, przygotowanie na przyszłą skalowalność
Repository Pattern	Abstrakcja dostępu do bazy danych	Testowalność, wymienność ORM bez zmiany logiki biznesowej
Domain Events	Komunikacja między modułami	Luźne powiązanie modułów, brak bezpośrednich zależności między nimi
Result Pattern	Obsługa błędów zamiast wyjątków	Jawna obsługa błędów, przewidywalne API, lepsza czytelność

2.2 Warstwy aplikacji

Warstwa	Projekt .csproj	Odpowiedzialność
Domain	Trivaxa.Domain	Encje, value objects, domain events, interfaces repozytoriów — zero zależności zewnętrznych
Application	Trivaxa.Application	Komendy CQRS, handlery MediatR, walidatory FluentValidation, DTOs, logika biznesowa
Infrastructure	Trivaxa.Infrastructure	EF Core DbContext, repozytoria, Hangfire, SMTP, integracje zewnętrzne
API	Trivaxa.API	Kontrolery ASP.NET Core, middleware, konfiguracja DI, Swagger, filtery wyjątków

2.3 Moduły systemu

System podzielony jest na sześć modułów, każdy z własną przestrzenią nazw, własnymi encjami i własnymi endpointami API:

Moduł	Przestrzeń nazw	Główne odpowiedzialności
Aparatura	Trivaxa.Modules.Equipment	Ewidencja urządzeń, przeglądy techniczne, naprawy, umowy serwisowe, gwarancje
Awarie	Trivaxa.Modules.Failures	Zgłoszenia awarii, statusy, historia incydentów, koszty interwencji

Moduł	Przestrzeń nazw	Główne odpowiedzialności
Serwisanci	Trivaxa.Modules.ServiceProviders	Baza firm serwisowych, kontrakty, dane kontaktowe
Powiadomienia	Trivaxa.Modules.Notifications	Harmonogram przypomnień, wysyłka email, historia powiadomień
Raporty	Trivaxa.Modules.Reports	Generowanie raportów PDF i XLSX, eksport danych RODO
Tożsamość	Trivaxa.Modules.Identity	Użytkownicy, role RBAC, tokeny JWT, audit log

3. Stos technologiczny

3.1 Decyzja: C# + ASP.NET Core 8

ASP.NET Core 8 jest wersją LTS (Long-Term Support) z gwarantowanym wsparciem Microsoft do 2026 roku i kolejnymi wersjami LTS co dwa lata. Oznacza to że system napisany dziś będzie wspierany bez konieczności przepisywania przez co najmniej dekadę.

Kryterium	Ocena dla ASP.NET Core 8	Komentarz
Stabilność API	Bardzo wysoka	Microsoft rzadko wprowadza breaking changes, ścieżka migracji zawsze dokumentowana
Wydajność	Najwyższa w benchmarkach	ASP.NET Core regularnie zajmuje 1. miejsce w TechEmpower benchmarks
Dojrzałość ekosystemu	Bardzo wysoka	EF Core, Hangfire, FluentValidation, MediatR — biblioteki z wieloletnią historią
Wbudowane możliwości	Kompletne	DI, middleware, konfiguracja, logowanie, health checks — all-in-one
Dokumentacja OpenAPI	Automatyczna	Swagger generowany z kodu, zero manualnej dokumentacji API
Testowanie	Doskonałe	xUnit, Moq, WebApplicationFactory do testów integracyjnych out-of-the-box
Docker na Debianie	Pełne wsparcie	Oficjalny obraz .NET Runtime dla Debian/Linux, arm64 i amd64

3.2 Kompletny stos zależności

Pakiet / Technologia	Wersja	Zastosowanie
ASP.NET Core	8.0 LTS	Framework HTTP, routing, middleware, DI
Entity Framework Core	8.0	ORM, migracje bazy danych, LINQ queries
Npgsql.EF.Core	8.0	Provider PostgreSQL dla EF Core
MediatR	12.x	CQRS — komendy, zapytania, domain events
FluentValidation	11.x	Walidacja requestów — deklaratywna, testowalna
Hangfire	1.8.x	Kolejka zadań cyklicznych i jednorazowych (przeglądy, powiadomienia)
Hangfire.PostgreSql	1.x	Backend kolejki Hangfire w PostgreSQL — bez Redisa
MailKit	4.x	Wysyłka email SMTP — profesjonalna, obsługa TLS
QuestPDF	2024.x	Generowanie raportów PDF — C# DSL, bez zależności zewnętrznych
ClosedXML	0.102.x	Generowanie plików XLSX — Excel bez instalacji Office
Serilog	3.x	Logowanie strukturalne — sink do pliku, konsoli i PostgreSQL

Pakiet / Technologia	Wersja	Zastosowanie
Scalar / Swashbuckle	latest	Dokumentacja OpenAPI / Swagger UI
xUnit + Moq	latest	Testy jednostkowe i integracyjne
Docker + Compose	latest	Konteneryzacja i orkiestracja na Debianie

4. Struktura projektu

4.1 Układ katalogów

Projekt zorganizowany zgodnie z zasadą Clean Architecture — zależności płyną wyłącznie do wewnątrz (API → Application → Domain):

```
trivaxa/
├── src/
│   ├── Trivaxa.API/                    # Warstwa wejściowa
│   │   ├── Controllers/                 # Kontrolery REST
│   │   │   ├── EquipmentController.cs
│   │   │   ├── FailuresController.cs
│   │   │   ├── NotificationsController.cs
│   │   │   ├── ReportsController.cs
│   │   │   └── IdentityController.cs
│   │   ├── Middleware/                  # Auth, error handling, logging
│   │   ├── Program.cs                  # Punkt wejścia, konfiguracja DI
│   │   └── appsettings.json
│   │
│   ├── Trivaxa.Application/             # Logika biznesowa
│   │   ├── Equipment/
│   │   │   ├── Commands/               # CreateDevice, UpdateDevice...
│   │   │   ├── Queries/                # GetDevices, GetDeviceById...
│   │   │   └── Validators/             # FluentValidation rules
│   │   ├── Failures/
│   │   ├── Notifications/
│   │   ├── Reports/
│   │   └── Identity/
│   │
│   ├── Trivaxa.Domain/                 # Serce systemu — zero zależności
│   │   ├── Entities/                   # Device, Failure, User...
│   │   ├── Enums/                      # DeviceStatus, FailurePriority...
│   │   ├── Events/                     # FailureReported, InspectionDue...
│   │   └── Interfaces/                 # IDeviceRepository, IUnitOfWork...
│   │
│   └── Trivaxa.Infrastructure/          # Implementacje techniczne
│       ├── Persistence/                # EF Core DbContext, migracje
│       ├── Repositories/               # Implementacje repozytoriów
│       ├── Jobs/                       # Hangfire jobs
│       ├── Email/                      # MailKit SMTP service
│       └── Reports/                    # QuestPDF + ClosedXML
│
└── tests/
    ├── Trivaxa.Unit.Tests/
    └── Trivaxa.Integration.Tests/
```

5. Model danych

5.1 Diagram encji — opis relacji

System operuje na trzech głównych grupach encji — aparatura, incydenty i tożsamość — powiązanych relacjami opisanymi poniżej:

Encja nadrzędna	Encja podrzędna	Relacja	Opis
Department (Oddział)	Device (Urządzenie)	1 : N	Jeden oddział posiada wiele urządzeń
Device (Urządzenie)	Inspection (Przegląd)	1 : N	Urządzenie ma historię przeglądów
Device (Urządzenie)	Failure (Awaria)	1 : N	Urządzenie może mieć wiele incydentów awarii
Failure (Awaria)	FailureStatusHistory	1 : N	Każda zmiana statusu awarii jest historyzowana
ServiceProvider (Serwisant)	Failure (Awaria)	1 : N	Firma serwisowa obsługuje wiele awarii
ServiceProvider (Serwisant)	ServiceContract (Kontrakt)	1 : N	Serwisant ma wiele kontraktów serwisowych
User (Użytkownik)	Failure (Awaria)	1 : N	Użytkownik zgłasza wiele awarii
User (Użytkownik)	AuditLog (Log audytu)	1 : N	Wszystkie operacje użytkownika są logowane

5.2 Kluczowe encje

Device — urządzenie medyczne

Pole	Typ C#	Typ SQL	Opis / Ograniczenia
Id	Guid	UUID PK	Klucz główny — GUID generowany przez aplikację
Name	string	VARCHAR(255) NOT NULL	Nazwa urządzenia
InventoryNumber	string	VARCHAR(100) UNIQUE NOT NULL	Numer inwentarzowy — unikalny w obrębie szpitala
SerialNumber	string?	VARCHAR(100)	Numer seryjny producenta (opcjonalny)
Manufacturer	string	VARCHAR(255) NOT NULL	Producent
Model	string	VARCHAR(255) NOT NULL	Model urządzenia
CategoryId	Guid	UUID FK → Category	Kategoria urządzenia (słownik konfigurowalny)
DepartmentId	Guid	UUID FK → Department	Oddział / lokalizacja

Pole	Typ C#	Typ SQL	Opis / Ograniczenia
Status	DeviceStatus (enum)	VARCHAR(30) NOT NULL	active | in_repair | archived | disposed
PurchaseDate	DateOnly?	DATE	Data zakupu
PurchasePrice	decimal?	NUMERIC(12,2)	Wartość zakupu
WarrantyExpires	DateOnly?	DATE	Data wygaśnięcia gwarancji
NextInspectionDate	DateOnly?	DATE	Termin kolejnego przeglądu (index)
Notes	string?	TEXT	Dodatkowe uwagi
CreatedAt	DateTime	TIMESTAMPTZ NOT NULL	Data dodania — ustawiana przez bazę
UpdatedAt	DateTime	TIMESTAMPTZ NOT NULL	Data modyfikacji — auto-update trigger
CreatedByUserId	Guid	UUID FK → User	Kto dodał urządzenie

Failure — zgłoszenie awarii

Pole	Typ C#	Typ SQL	Opis / Ograniczenia
Id	Guid	UUID PK	Klucz główny
DeviceId	Guid	UUID FK → Device NOT NULL	Urządzenie, którego dotyczy awaria
ReportedByUserId	Guid	UUID FK → User NOT NULL	Zgłaszający
DepartmentId	Guid	UUID FK → Department NOT NULL	Oddział zgłaszający
Description	string	TEXT NOT NULL	Opis awarii — wymagany
Status	FailureStatus (enum)	VARCHAR(30) NOT NULL	open | in_progress | waiting_parts | resolved | closed
Priority	FailurePriority (enum)	VARCHAR(20) NOT NULL	low | medium | high | critical
ServiceProviderId	Guid?	UUID FK → ServiceProvider	Przypisana firma serwisowa (opcjonalne)
RepairCost	decimal?	NUMERIC(12,2)	Koszt naprawy — uzupełniany po zakończeniu
ResolvedAt	DateTime?	TIMESTAMPTZ	Data i czas rozwiązania
CreatedAt	DateTime	TIMESTAMPTZ NOT NULL	Data i czas zgłoszenia

User — użytkownik systemu

Pole	Typ C#	Typ SQL	Opis / Ograniczenia
Id	Guid	UUID PK	Klucz główny
Email	string	VARCHAR(255) UNIQUE NOT NULL	Login — adres email
PasswordHash	string	VARCHAR(500) NOT NULL	Hash bcrypt (work factor 12)
FirstName	string	VARCHAR(100) NOT NULL	Imię
LastName	string	VARCHAR(100) NOT NULL	Nazwisko
Role	UserRole (enum)	VARCHAR(50) NOT NULL	equipment_worker | equipment_manager | nurse | administrator
DepartmentId	Guid?	UUID FK → Department	Oddział (wymagany dla pielęgniarki)
IsActive	bool	BOOLEAN NOT NULL DEFAULT true	Soft-disable konta bez usuwania
LastLoginAt	DateTime?	TIMESTAMPTZ	Data ostatniego logowania
ActivationToken	string?	VARCHAR(500)	Token aktywacyjny (jednorazowy, TTL 72h)
CreatedAt	DateTime	TIMESTAMPTZ NOT NULL	Data utworzenia konta

6. Specyfikacja REST API

6.1 Konwencje

Konwencja	Wartość
Base URL	https://{host}/api/v1
Format danych	JSON (Content-Type: application/json)
Uwierzytelnianie	Bearer token JWT w nagłówku Authorization
Kody sukcesu	200 OK, 201 Created, 204 No Content
Kody błędów	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable, 500 Internal
Paginacja	?page=1&pageSize=25 — domyślnie 25 rekordów, max 100
Sortowanie	?sortBy=name&sortDir=asc|desc
Filtrowanie	?status=active&departmentId={guid}
Wersjonowanie	Prefiks /api/v1 — zmiany breaking w nowej wersji /api/v2

6.2 Moduł: Aparatura — /api/v1/devices

Metoda	Endpoint	Rola	Opis
GET	/devices	worker, manager, admin	Lista urządzeń z filtrowaniem, sortowaniem i paginacją
GET	/devices/{id}	worker, manager, nurse, admin	Szczegóły pojedynczego urządzenia
GET	/devices/department/{deptId}	nurse, worker, manager	Urządzenia przypisane do oddziału
POST	/devices	worker, manager	Dodanie nowego urządzenia
PUT	/devices/{id}	worker, manager	Aktualizacja danych urządzenia
PATCH	/devices/{id}/status	manager	Zmiana statusu (np. disposed wymaga roli manager)
DELETE	/devices/{id}	manager	Soft-delete z archiwizacją (nie usuwa z bazy)
GET	/devices/{id}/inspections	worker, manager	Historia przeglądów urządzenia
POST	/devices/{id}/inspections	worker, manager	Dodanie wpisu o przeglądzie
GET	/devices/{id}/failures	worker, manager	Historia awarii urządzenia

6.3 Moduł: Awarie — /api/v1/failures

Metoda	Endpoint	Rola	Opis
GET	/failures	worker, manager, admin	Lista awarii z filtrowaniem po statusie, priorytecie, oddziale
GET	/failures/{id}	worker, manager, nurse	Szczegóły zgłoszenia awarii
POST	/failures	nurse, worker, manager	Zgłoszenie nowej awarii — wyzwala domain event
PATCH	/failures/{id}/status	worker, manager	Zmiana statusu awarii
PATCH	/failures/{id}/assign	worker, manager	Przypisanie firmy serwisowej
PATCH	/failures/{id}/resolve	worker, manager	Zamknięcie awarii z kosztem i opisem naprawy
GET	/failures/{id}/history	worker, manager	Historia zmian statusu

6.4 Moduł: Tożsamość — /api/v1/auth

Metoda	Endpoint	Rola	Opis
POST	/auth/login	publiczny	Logowanie — zwraca access token (15 min) + refresh token (7 dni)
POST	/auth/refresh	publiczny	Odświeżenie tokenu — wymaga ważnego refresh tokenu
POST	/auth/logout	zalogowany	Unieważnienie refresh tokenu
POST	/auth/activate	publiczny	Aktywacja konta tokenem jednorazowym
POST	/users	admin	Utworzenie konta + wygenerowanie tokenu aktywacyjnego
GET	/users	admin	Lista użytkowników
PUT	/users/{id}/role	admin	Zmiana roli użytkownika
PATCH	/users/{id}/deactivate	admin	Dezaktywacja konta

6.5 Moduł: Raporty — /api/v1/reports

Metoda	Endpoint	Rola	Opis
POST	/reports/equipment	worker, manager	Raport aparatury — lista urządzeń z parametrami (PDF/XLSX)
POST	/reports/failures	worker, manager	Raport awarii z kosztami w zakresie dat (PDF/XLSX)

Metoda	Endpoint	Rola	Opis
POST	/reports/inspections	worker, manager	Raport przeglądów — zrealizowane i zaplanowane (PDF/XLSX)
GET	/reports/export/gdpr/{userId}	admin	Eksport danych użytkownika zgodny z RODO

7. Bezpieczeństwo

7.1 Uwierzytelnianie JWT

System stosuje dwuetapowe tokeny JWT bez stanu po stronie serwera:

Token	TTL	Przechowywanie	Zastosowanie
Access Token	15 minut	Pamięć klienta (nie localStorage)	Wysyłany w nagłówku Authorization: Bearer {token}
Refresh Token	7 dni	HttpOnly Secure Cookie	Wymiana na nowy access token — niepodatny na XSS

Payload tokenu JWT zawiera: userId, email, role, departmentId, iat, exp. Klucz podpisujący (RS256) przechowywany w zmiennej środowiskowej — nigdy w kodzie ani repozytorium.

7.2 RBAC — macierz uprawnień

Operacja	Nurse	Worker	Manager	Admin
Podgląd urządzeń własnego oddziału	TAK	TAK	TAK	TAK
Podgląd wszystkich urządzeń	NIE	TAK	TAK	TAK
Dodanie/edycja urządzenia	NIE	TAK	TAK	TAK
Kasacja urządzenia (soft-delete)	NIE	NIE	TAK	TAK
Zgłoszenie awarii	TAK	TAK	TAK	NIE
Obsługa awarii / zmiana statusu	NIE	TAK	TAK	NIE
Generowanie raportów	NIE	TAK	TAK	NIE
Zarządzanie użytkownikami	NIE	NIE	NIE	TAK
Migracja bazy danych	NIE	NIE	NIE	TAK
Konfiguracja słowników	NIE	NIE	TAK	TAK

7.3 Audit Log

Każda operacja modyfikacji lub usunięcia danych zapisywana jest do tabeli AuditLog z następującymi polami:

- **UserId** — kto wykonał operację
- **Action** — CREATE | UPDATE | DELETE
- **EntityType** — nazwa encji (np. Device, Failure)
- **EntityId** — identyfikator zmienionej encji
- **OldValues** — JSON ze stanem przed zmianą
- **NewValues** — JSON ze stanem po zmianie

- **IpAddress** — adres IP klienta
- **CreatedAt** — timestamp operacji UTC

Audit log jest tylko do dczytu — brak endpointu modyfikującego. Zapis realizowany przez interceptor EF Core, nie przez kod aplikacji, co eliminuje możliwość pominięcia logowania.

8. Kolejka zadań — Hangfire

8.1 Dlaczego Hangfire zamiast Redisa

Hangfire używa PostgreSQL jako backend kolejki — tej samej bazy danych co cała aplikacja. Eliminuje to potrzebę osobnego kontenera Redis, upraszcza infrastrukturę i backup (jedna baza do zabezpieczenia). Wydajność jest w pełni wystarczająca dla skali szpitalnej (dziesiątki zadań dziennie, nie miliony).

8.2 Zdefiniowane zadania cykliczne

Zadanie (Job)	Harmonogram (CRON)	Opis działania
InspectionDueNotificationJob	0 7 * * * (codziennie 7:00)	Pobiera urządzenia z NextInspectionDate ≤ dziś + 30 dni. Wysyła email do Pracownika i Kierownika DAM. Jeśli data minęła bez wpisu — ponawia co 7 dni.
WarrantyExpirationJob	0 8 * * * (codziennie 8:00)	Pobiera urządzenia z WarrantyExpires ≤ dziś + 14 dni. Wysyła email do Kierownika DAM.
FailureNotificationJob	Wyzwalany zdarzeniem	Natychmiastowe powiadomienie po zgłoszeniu awarii. Wysyłany do Pracownika DAM w < 60 sekund od zgłoszenia.
DatabaseBackupJob	0 2 * * * (codziennie 2:00)	Wywołuje pg_dump i zapisuje dump do /var/trivaxa/backups/. Retencja: 30 plików.
AuditLogCleanupJob	0 3 1 * * (1. dnia miesiąca)	Archiwizuje logi audytu starsze niż 2 lata do tabeli archiwalnej.

Hangfire Dashboard dostępny pod /hangfire (zabezpieczony — wyłącznie rola admin). Dashboard pokazuje historię zadań, błędy i możliwość ręcznego wyzwolenia.

9. Infrastruktura — Debian + Docker

9.1 Wymagania sprzętowe serwera

Komponent	Minimum	Rekomendowane	Uzasadnienie
CPU	2 rdzenie	4 rdzenie	ASP.NET Core jest wielowątkowy — więcej rdzeni = lepsza obsługa równoczesnych żądań
RAM	4 GB	8 GB	.NET Runtime + PostgreSQL + Hangfire + bufor OS — 4 GB to absolutne minimum
Dysk	100 GB SSD	500 GB SSD	Baza danych + logi + backupy. SSD wymagany dla wydajności PostgreSQL
System operacyjny	Debian 12 Bookworm	Debian 12 Bookworm	Stabilna gałąź LTS, wsparcie do 2028
Sieć	100 Mbit LAN	1 Gbit LAN	Ruch wewnętrzny szpitala — 100 Mbit wystarczy
UPS	Wymagany	Wymagany	System medyczny — ochrona przed utratą danych przy zaniku zasilania

9.2 Docker Compose — konfiguracja usług

```
# docker-compose.yml
version: '3.9'

services:
  api:
    image: trivaxa-api:latest
    build: ./src/Trivaxa.API
    restart: always
    ports: ['5000:8080']
    environment:
      -
ConnectionStrings__Default=Host=db;Database=trivaxa;Username=trivaxa;Password=$
{DB_PASS}
      - Jwt__SecretKey=${JWT_SECRET}
      - Smtp__Host=${SMTP_HOST}
    depends_on: [db]
    volumes: ['/var/trivaxa/logs:/app/logs']

  db:
    image: postgres:16-alpine
    restart: always
    environment:
      POSTGRES_DB: trivaxa
      POSTGRES_USER: trivaxa
      POSTGRES_PASSWORD: ${DB_PASS}
    volumes:
      - '/var/trivaxa/postgres:/var/lib/postgresql/data'
      - '/var/trivaxa/backups:/backups'
    ports: ['5432:5432']   # tylko w sieci wewnętrznej, UFW blokuje z zewnątrz
```

```
  nginx:
    image: nginx:alpine
    restart: always
    ports: ['443:443', '80:80']
    volumes:
      - './nginx/nginx.conf:/etc/nginx/nginx.conf:ro'
      - '/etc/letsencrypt:/etc/letsencrypt:ro'
    depends_on: [api]
```

9.3 Konfiguracja Nginx (reverse proxy)

```
# nginx/nginx.conf
server {
    listen 443 ssl http2;
    server_name trivaxa.szpital.local;

    ssl_certificate
/etc/letsencrypt/live/trivaxa.szpital.local/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/trivaxa.szpital.local/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    # Rate limiting
    limit_req_zone $binary_remote_addr zone=api:10m rate=30r/m;
    limit_req zone=api burst=10 nodelay;

    location /api/ {
        proxy_pass         http://api:8080;
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
    }

    # Frontend (zostanie dodany po decyzji o technologii)
    location / {
        proxy_pass http://frontend:3000;
    }
}

# Redirect HTTP -> HTTPS
server { listen 80; return 301 https://$host$request_uri; }
```

9.4 Systemd — autostart

```
# /etc/systemd/system/trivaxa.service
[Unit]
Description=Trivaxa Medical Equipment System
Requires=docker.service
After=docker.service network-online.target
```

```
[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/trivaxa
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target

# Aktywacja: systemctl enable trivaxa && systemctl start trivaxa
```

9.5 UFW Firewall — polityka portów

Port	Protokół	Status	Uzasadnienie
22	TCP	OPEN (ograniczony do IP admina)	SSH do zarządzania serwerem
80	TCP	OPEN	HTTP — redirect do HTTPS przez Nginx
443	TCP	OPEN	HTTPS — jedyny port aplikacyjny widoczny z sieci
5432	TCP	ZABLOKOWANY z zewnątrz	PostgreSQL dostępny tylko wewnątrz Docker network
8080	TCP	ZABLOKOWANY z zewnątrz	ASP.NET Core API — wyłącznie przez Nginx proxy
Pozostałe	ALL	ZABLOKOWANE	Domyślna polityka deny-all

10. Migracja danych

10.1 Import z Excel

Jednorazowa migracja danych aparatury z istniejących arkuszy Excel realizowana przez dedykowany endpoint administratora:

1. Administrator przesyła plik Excel (.xlsx) przez endpoint POST /api/v1/admin/import/equipment
2. System weryfikuje format pliku i mapuje kolumny na pola encji Device (konfigurowalny mapping)
3. Dane przechodzą przez walidację FluentValidation — błędne wiersze trafiają do raportu błędów
4. Administrator otrzymuje raport: X rekordów zaimportowanych, Y błędnych (z opisem), Z pominętych duplikatów
5. Dopiero po zatwierdzeniu przez administratora dane są zapisywane do bazy (transakcja)
6. Cały proces jest logowany w audit logu

Zasada	System wywołuje wyłącznie migrację — nie nadpisuje istniejących danych. Każda re-migracja wymaga świadomej decyzji administratora i jest osobną transakcją z pełnym logiem.

10.2 Migracje schematu bazy (EF Core)

Schemat bazy danych zarządzany jest przez migracje EF Core — nie przez ręczne skrypty SQL:

```
# Tworzenie nowej migracji (deweloperzy)
dotnet ef migrations add AddDeviceNotes --project Trivaxa.Infrastructure

# Aplikacja migracji (wdrożenie)
dotnet ef database update

# Lub automatycznie przy starcie aplikacji (appsettings.json)
"Database": { "AutoMigrate": true }

# Rollback do poprzedniej migracji
dotnet ef database update PreviousMigrationName
```

11. Monitoring i backup

11.1 Health checks

ASP.NET Core ma wbudowane health checks dostępne pod `/health`. System sprawdza:

- Połączenie z bazą danych PostgreSQL
- Dostępność kolejki Hangfire
- Połączenie z serwerem SMTP
- Dostępność miejsca na dysku (alert przy < 10 GB)

Nginx sprawdza `/health` co 30 sekund. Przy błędzie — loguje i wysyła alert email do administratora.

11.2 Logowanie (Serilog)

Poziom	Sink (cel)	Zawartość
Information	Plik + konsola Docker	Każde żądanie HTTP: metoda, ścieżka, status, czas odpowiedzi
Warning	Plik + konsola Docker	Błędy walidacji, nieudane logowania, brakujące zasoby
Error	Plik + konsola Docker	Wyjątki aplikacji, błędy bazy danych
Audit (custom)	Tabela AuditLog w PostgreSQL	Operacje CRUD na encjach — pełna historia zmian

11.3 Strategia backupu

Typ backupu	Częstotliwość	Retencja	Lokalizacja
Pełny dump PostgreSQL	Codziennie o 2:00	30 dni	/var/trivaxa/backups/ na serwerze
Przyrostowy dump	Co 6 godzin	7 dni	/var/trivaxa/backups/ na serwerze
Backup off-site	Codziennie o 3:00	90 dni	Szyfrowany (AES-256) transfer na NAS szpitala lub drugi serwer
Backup konfiguracji	Przy każdej zmianie	Bezterminowo	Repozytorium Git (bez danych wrażliwych)

Krytyczne	Backup na tym samym dysku co dane to nie backup. Minimum: drugi dysk (RAID 1) lub transfer na NAS szpitala. Należy ustalić z działem IT szpitala dostępną infrastrukturę przed wdrożeniem.

12. Harmonogram implementacji backendu

12.1 Fazy i kamienie milowe

Faza	Zakres prac	Czas	Kamień milowy
0 — Setup	Repozytorium Git, struktura projektu Clean Architecture, konfiguracja Docker Compose na Debianie deweloperskim, CI/CD (GitHub Actions: build + testy), schemat bazy + pierwsze migracje EF Core	2 tygodnie	Działający kontener API z endpointem /health
1 — Tożsamość i RBAC	Moduł Identity: rejestracja przez token admina, login JWT, refresh token, RBAC middleware, audit log interceptor EF Core	2 tygodnie	Działające uwierzytelnianie — testy integracyjne przechodzą
2 — Aparatura	Moduł Equipment: pełny CRUD urządzeń, przeglądy, migracja z Excel, walidacja FluentValidation, paginacja i filtrowanie	3 tygodnie	Endpoint /api/v1/devices w pełni funkcjonalny z testami
3 — Awarie	Moduł Failures: zgłoszenia, statusy, historia, przypisanie serwisanta, domain events, koszty	2 tygodnie	Pełny przepływ awarii od zgłoszenia do zamknięcia
4 — Powiadomienia	Moduł Notifications: Hangfire jobs, MailKit SMTP, szablony email, testy powiadomień end-to-end	2 tygodnie	Automatyczne powiadomienia o przeglądach działają
5 — Raporty	Moduł Reports: QuestPDF (PDF), ClosedXML (XLSX), eksport RODO, Hangfire dashboard	2 tygodnie	Generowanie raportów w < 10 sekund
6 — Serwisanci i konfiguracja	Moduł ServiceProviders, słowniki konfiguracyjne (Categories, Departments), panel admina	2 tygodnie	Kompletny backend API gotowy do integracji
7 — Testy i hardening	Testy penetracyjne, testy wydajnościowe (50+ równoczesnych użytkowników), audyt bezpieczeństwa, dokumentacja Swagger kompletna	3 tygodnie	Raport testów, API gotowe na podłączenie frontendu

Łączny czas implementacji backendu: 15-18 tygodni (2 osoby, równoległe zadania tam gdzie możliwe = skrócenie czasu).

Po ukończeniu Fazy 7 backend będzie w pełni funkcjonalny i udokumentowany. Dopiero w tym momencie zostanie podjęta ostateczna decyzja o technologii frontendowej — na podstawie rzeczywistej złożoności endpointów i wymagań widoków użytkownika.